



* 1. What is Docker & Containers ?

- **Definition:** Open platform for developing, shipping, and running applications anywhere.
- **Containers:** Lightweight, standalone, executable packages with code, runtime, and tools.
- **Containers vs VMs:** Share host OS kernel, start in milliseconds, consume 10x less RAM.
- **Portability:** 'Build once, run anywhere' across dev, staging, and production.

* Docker Core Architecture:

Docker Client - CLI tool (docker run)

Docker Daemon - dockerd manages containers

Images - Read-only blueprint layers

Containers - Running image instances

DOCKER CLIENT

- docker build
- docker pull
- docker run

REST API

REGISTRY (Docker Hub)

- Public/Private Repos

DOCKER HOST (Daemon)

- Images & Running Containers

* Hello World Docker Command:

```
# Pull image and run an interactive container with port
docker run -d -p 8080:80 --name my-web nginx:alpine

# Verify running container status
docker ps
```



DOCKER CHEAT SHEET

Part - 2: Essential CLI Commands

Docker Notes

* 2. Must-Know Docker Commands :

- **Container Lifecycle:** Commands to create, execute, inspect, and remove containers.
- **Image Operations:** Commands to build from Dockerfiles and pull from Docker Hub.
- **Housekeeping:** Pruning dangling images and reclaiming system disk space.

Command	Description	Example
<code>docker run</code>	Create & start container	<code>docker run -p 3000:3</code>
<code>docker ps -a</code>	List all containers	<code>docker ps -a --forma</code>
<code>docker exec</code>	Run command in container	<code>docker exec -it web</code>
<code>docker logs</code>	Fetch container stdout	<code>docker logs -f --tai</code>
<code>docker build</code>	Build image from Dockerfile	<code>docker build -t api:</code>
<code>docker stop/rm</code>	Stop and delete container	<code>docker stop web && d</code>

* Pro Docker CLI Aliases & Cleanup:

Clean all stopped containers, unused networks, and dan

`docker system prune -af --volumes`

Inspect live container resource usage (CPU/Memory)

`docker stats --no-stream`



* 3. Writing Efficient Dockerfiles :

- **Layer Caching:** Place infrequently changed instructions (COPY package.json) first.
- **Minimal Base Images:** Always use Alpine or Slim variants for small attack surface.
- **Non-Root User:** Run processes under a dedicated user for production security.
- **.dockerignore:** Exclude node_modules, .git, and temporary files from context.

* Multi-Stage Production Dockerfile Example:

```
# Stage 1: Build binary / dependencies
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Stage 2: Minimal runtime container
FROM python:3.11-slim
```



* 4. Docker Compose & Persistent Volumes :

- **Compose:** YAML tool for defining and running multi-container Docker applications.
- **Networking:** Containers in the same compose file communicate via service name as DNS.
- **Volumes:** Decouples persistent data (e.g. Postgres DB data) from container lifecycles.

* *docker-compose.yml Production Stack:*

```
version: '3.8'
services:
  web:
    build: .
    ports: ['8000:8000']
    depends_on: ['db']
    environment:
      - DATABASE_URL=postgres://user:pass@db:5442/app
```